C O L É G I O UNASP

POO – Programação Orientada a Objetos

Curso Técnico em Informática Integrado ao Ensino Médio

Turmas: 2TIA · 2TIB · 2TIC · 2TID **Bimestre:** 3º Bimestre · 2026 **Professor(a):**

RECUPERAÇÃO – PROJETO INTEGRADOR | VALOR: 10,0 PONTOS

Objetivo

Demonstrar domínio completo do ciclo de análise, modelagem e implementação orientada a objetos, desenvolvendo um sistema de maior complexidade, com múltiplas hierarquias de herança, relacionamentos variados e documentação UML abrangente — incluindo três Diagramas de Sequência.

Especificação do Sistema – Escola de Idiomas "Língua Viva"

⬜ Contexto

A Escola de Idiomas "Língua Viva" oferece cursos de inglês, espanhol e francês em três níveis (Básico, Intermediário e Avançado). Com o crescimento da escola, o controle manual de matrículas, pagamentos e frequência se tornou inviável.

A coordenadora pedagógica solicitou ao setor de TI um sistema via terminal capaz de gerenciar alunos, professores, turmas, matrículas, pagamentos mensais e registro de frequência em aulas.

Informações levantadas com a coordenadora:

Categoria	Informação coletada
Pessoas	O sistema possui três tipos de usuário: Aluno (possui nível de proficiência atual e idioma de interesse), Professor (possui lista de idiomas que leciona e carga horária semanal) e Coordenador (pode cancelar matrículas e emitir relatórios gerenciais). Todos têm: nome, CPF, telefone e e-mail.
Cursos e Turmas	Um Curso define: idioma, nível e valor da mensalidade. Uma Turma vincula um Curso a um Professor, define o horário (dia da semana + hora) e tem capacidade máxima de alunos. Uma turma só pode ser criada se houver um professor habilitado no idioma do curso.
Matrículas	Uma Matrícula vincula um Aluno a uma Turma e possui três estados: Ativa, Suspensa (aluno inadimplente por mais de 30 dias) e Cancelada. Uma matrícula é cancelada apenas pelo Coordenador. Ao ser cancelada, não pode mais ser reativada.

Pagamentos	Cada mês, um Pagamento é gerado automaticamente para cada matrícula ativa. Possui: mês de referência, valor (igual à mensalidade do curso) e status: Pendente, Pago ou Atrasado (quando vence sem quitação). Ao registrar um pagamento, o sistema verifica se há outros meses em atraso e os sinaliza.
Aulas e Frequência	Cada Aula pertence a uma Turma e registra: data, conteúdo ministrado e lista de presenças. Ao registrar a frequência, o sistema verifica se o aluno está com matrícula Ativa — alunos com matrícula Suspensa não podem ter presença registrada até regularizarem o pagamento.
Relatórios	O sistema deve gerar: (a) lista de turmas ativas com vagas disponíveis; (b) alunos inadimplentes (matrículas suspensas); (c) frequência de um aluno em uma turma; (d) histórico de pagamentos de um aluno por CPF.
Tratamento de erros	Tratar com exceções personalizadas: turma sem vagas disponíveis, professor não habilitado no idioma do curso, aluno já matriculado na mesma turma, tentativa de registrar frequência com matrícula suspensa, operação inválida em matrícula cancelada.

Dica de análise: *Identifique os substantivos (Pessoa, Aluno, Professor, Coordenador, Curso, Turma, Matricula, Pagamento, Aula, Frequencia) e os verbos (matricular, pagar, suspender, cancelar, registrar frequência, emitir relatório). Atenção ao ciclo de vida da Matrícula — três estados com transições — e às hierarquias de herança (Pessoa com três subclasses).*

Entregáveis

1. **Diagrama de Casos de Uso (Use Case Diagram):** Identificar todos os atores (Aluno, Professor, Coordenador) e os casos de uso do sistema. Relacionamentos include e extend são obrigatórios onde aplicável — por exemplo, "Registrar Frequência" inclui a verificação do status da matrícula.
2. **Diagrama de Classes UML completo:** Todas as classes com atributos (visibilidade e tipo), métodos (visibilidade e assinatura), relacionamentos (herança, associação, composição, agregação) e multiplicidades corretas. Devem constar obrigatoriamente: a hierarquia de Pessoa, a composição entre Turma e Aula, a associação entre Matricula e Pagamento, e a enumeração de estados (MatriculaStatus e PagamentoStatus).
3. **Diagramas de Sequência — 3 obrigatórios:** Os três fluxos abaixo devem ser documentados individualmente, cada um em seu próprio diagrama, com todos os objetos participantes, mensagens trocadas e retornos representados:

Sequência 1 — Matricular Aluno em Turma

Representar: verificação de vagas na turma, verificação se o aluno já está matriculado, criação da Matrícula com status Ativa e geração automática do primeiro Pagamento com status Pendente.

Sequência 2 — Registrar Pagamento e Atualizar Status da Matrícula

Representar: busca do pagamento pelo mês de referência, verificação de outros pagamentos em atraso, marcação do pagamento como Pago e, se aplicável, reativação automática da Matrícula de Suspensa para Ativa.

Sequência 3 — Registrar Frequência em Aula

Representar: busca da Aula e do Aluno, verificação do status da Matrícula (lançar exceção se Suspensa ou Cancelada), criação do registro de Frequência com presença ou ausência e atualização da contagem de faltas.

4. **Código Python — implementação completa:** Implementação de todas as classes a partir do Diagrama de Classes, incluindo:
   - Hierarquia de herança de Pessoa (Aluno, Professor, Coordenador)
   - Encapsulamento com atributos privados e @property
   - Enumerações de estado com enum.Enum (MatriculaStatus, PagamentoStatus)
   - Todas as exceções personalizadas criadas e tratadas
   - Lógica de suspensão automática de matrícula por inadimplência
   - Menu interativo no terminal cobrindo todas as funcionalidades
5. **Apresentação oral ao professor:** Demonstração do sistema em funcionamento, com ênfase na explicação do ciclo de vida da Matrícula, nas decisões de design das exceções e na correspondência entre o Diagrama de Classes e o código implementado. O professor poderá solicitar alterações ao vivo durante a apresentação.

⚠️ **Atenção:** Os diagramas devem ser entregues em formato digital (.png, .jpg, .pdf ou arquivo de ferramenta UML) ou desenhados a mão e fotografados com boa qualidade. A implementação Python deve ser um arquivo .py funcional. Envie tudo via e-class ou Google Classroom, conforme orientação do professor.

Integridade Acadêmica – Plágio e Uso de Inteligência Artificial

Esta avaliação exige produção intelectual própria. O uso de ferramentas de Inteligência Artificial (como ChatGPT, GitHub Copilot, Gemini, Claude e similares) para gerar o código, os diagramas ou qualquer parte da documentação é considerado uso indevido e caracteriza desonestidade acadêmica.

Da mesma forma, copiar total ou parcialmente o trabalho de outro aluno — ou permitir que o próprio trabalho seja copiado — constitui plágio.

Evidências de plágio ou uso indevido de IA resultarão em nota zero para todos os envolvidos, **sem direito a nova avaliação,** e poderão resultar em encaminhamento à coordenação pedagógica para as providências cabíveis.

O uso de IA como ferramenta de aprendizagem ***é permitido e incentivado*** *— para tirar dúvidas, entender conceitos e revisar o que foi estudado. O que não é permitido é entregar como seu algo que foi produzido por ela.*

Observações Importantes

1. **Trabalho individual.** O código e os diagramas devem ser de autoria própria. 2. **O código deve executar sem erros** no momento da apresentação. Teste antes de entregar. 3. **Os três Diagramas de Sequência são obrigatórios.** A ausência de qualquer um deles implica perda integral dos pontos do critério correspondente.
4. **Nomenclatura:** use português ou inglês de forma consistente. Convenções Python: classes em PascalCase, atributos e métodos em snake_case.
5. **Boas práticas:** nomes significativos, responsabilidade única por classe, sem código duplicado desnecessário.
6. **Ferramentas de UML sugeridas (gratuitas):** draw.io (diagrams.net), Lucidchart, StarUML ou mesmo papel e caneta (foto nítida).
7. **Dúvidas** devem ser tiradas com o professor durante a aula de recuperação.

Material de estudo: apostilas e slides disponibilizados no e-class e Google Classroom (1º e 2º bimestres: fundamentos Python, encapsulamento, herança, polimorfismo, exceções, UML introdutório; 3º bimestre: análise de requisitos, Casos de Uso, Diagrama de Classes completo, Diagrama de Sequência, implementação a partir do diagrama).